French University in Armenia

Faculty of Computer Science and Applied Mathematics

Final report

Project S4

“Student Performance Tracking System with OOP and Database Integration**”**

Presented by the 3rd academic year students: Margarita Sargsyan

Mariam Harutyunyan

Submission date: May 2

Yerevan 2026

Summary

This project presents the development of a Student Performance Management and Recommendation System using Java object-oriented programming and SQL database technologies. The main objective of the system is to analyze student academic performance and provide personalized recommendations based on predefined rules.

The system collects and stores student data, including quiz scores, assignment results, and activity levels, in a structured SQL database. Using Java OOP principles, the system processes this data through different classes and applies rule-based logic to identify weak areas. Based on this analysis, it generates recommendations to help students improve their performance.

The scope of the project includes database design, implementation of object-oriented structures, and development of a simple logic-based recommendation engine. Unlike machine learning systems, this project focuses on transparency, simplicity, and efficient data handling.

The current outcome is a functional prototype that demonstrates how core computer science concepts such as OOP, databases, and system design can be integrated to solve real-world educational problems. The project shows the practical application of software engineering principles in building a complete and structured system.

Contents

Introduction

Project Background and Context

In recent years, educational systems have become increasingly digital, allowing institutions to collect large amounts of student data through quizzes, assignments, online platforms, and learning management systems. This data contains valuable information about student performance, learning behavior, and progress over time. However, in many cases, this data is not fully utilized. Most educational systems still rely on traditional evaluation methods, where students receive only final grades or general feedback without detailed explanations of their weaknesses.

As a result, students often struggle to understand which specific topics they need to improve. They may repeat the same mistakes without clear guidance, which slows down their learning process. At the same time, instructors face challenges in analyzing large numbers of students individually, especially in courses with many participants. This creates a gap between available data and its practical use in improving education.

With the advancement of software engineering, it is possible to develop systems that can organize, process, and analyze student data efficiently. Technologies such as object-oriented programming and relational databases provide strong tools for building structured and scalable systems. These technologies allow developers to design modular systems, store large datasets, and retrieve information quickly using optimized queries.

This project is developed within this context. It focuses on creating a system that uses Java object-oriented programming and SQL database management to handle student performance data. Instead

of relying on complex machine learning models, the system applies rule-based logic to ensure transparency and simplicity. The goal is to demonstrate how fundamental computer science concepts can be applied to build a practical and efficient educational support system.

Problem Statement and Objectives

Problem Statement

In modern educational environments, students are evaluated through quizzes, assignments, and exams, but the feedback they receive is often limited to numerical grades. These grades do not clearly explain which specific topics the student does not understand or why their performance is low. As a result, students may feel confused about how to improve and may continue making the same mistakes without targeted guidance.

Another important issue is that instructors are often responsible for a large number of students, which makes it difficult to analyze each student's performance individually. Although a significant amount of data is collected, such as scores and activity records, it is rarely processed in a structured and efficient way. This leads to underutilization of valuable information that could otherwise help improve learning outcomes.

In addition, many existing systems do not provide a simple and transparent way to analyze performance data. Some advanced systems use complex techniques that are not always easy to understand or implement. Therefore, there is a need for a system that can process student data, identify weak areas, and provide clear recommendations in a simple and understandable manner.

This project addresses these problems by developing a structured system that uses rule-based logic, object-oriented programming, and database technologies to analyze student performance and support learning improvement.

Project Objectives

The main objective of this project is to design and implement a Student Performance Management and Recommendation System that can analyze academic data and provide useful feedback to students. The system aims to apply fundamental computer science concepts to solve a real-world educational problem.

The specific objectives of the project are the following:

- To design the system using Java object-oriented programming principles, ensuring modularity and clear structure
- To create and manage a relational database using SQL Server for storing student information
- To collect and organize student performance data, including quiz scores, assignment results, and activity levels
- To develop a rule-based logic system that analyzes data using predefined conditions
- To identify weak areas in student performance based on these conditions
- To generate personalized and easy-to-understand recommendations for improvement
- To ensure that the system is efficient, simple, and transparent
- To demonstrate the integration of programming, database management, and system design concepts
- To build a functional prototype that can be extended in future developments

By achieving these objectives, the project demonstrates how software engineering techniques can be used to create a practical system that supports both students and educators.

Description of Target Audience and End Users

The Student Performance Tracking and Recommendation System is designed to serve several groups of users, each with different roles and benefits. The system focuses on transforming raw academic data into meaningful information that can support learning and decision-making.

The primary target audience of the system is students. Students are the main users who interact directly with the system to monitor their academic performance. By viewing their quiz scores, assignment results, and activity levels, students can better understand their strengths and weaknesses. The system provides clear recommendations based on rule-based analysis, helping students identify which topics require additional practice. This allows them to study more efficiently and focus on areas that need improvement instead of spending time on already mastered topics.

The system is designed to be simple and easy to use, ensuring accessibility for students with different levels of technical knowledge. The interface and recommendations are presented in a clear and structured way, allowing users to quickly interpret the results and apply them to their learning process. This supports independent learning and helps students take responsibility for their academic development.

The secondary group of users includes instructors and teachers. Although they may not use the system as frequently as students, it provides them with valuable insights into student performance. By analyzing the data, instructors can identify common patterns, such as topics where many students perform poorly. This information allows teachers to adjust their teaching methods, provide additional explanations, and improve the overall learning experience. The system also reduces the need for manual analysis, saving time and effort.

Educational institutions and administrators represent another important group of end users. They can use the system to monitor overall academic performance and identify trends across courses or groups of students. This information can support strategic decisions, such as improving curriculum design or allocating resources more effectively. In addition, the system can be extended in the future to integrate with existing learning management systems, making it suitable for larger-scale use.

Finally, the system can also be useful for developers and researchers interested in educational technologies. It demonstrates how core computer science concepts, such as object-oriented programming and database integration, can be applied to solve real-world problems in education.

Overall, the system is designed to be flexible, scalable, and user-friendly. It provides value to students, instructors, and institutions by converting academic data into useful insights and actionable recommendations, thereby improving the learning process and supporting better educational outcomes.

Outline of Report Structure

This report is organized into several structured sections, each explaining a specific aspect of the Student Performance Tracking and Recommendation System. The purpose of this structure is to provide a clear and logical presentation of the project, from the initial idea to the final implementation and evaluation. Each section contributes to a complete understanding of both the technical and conceptual development of the system.

The report begins with the **Summary**, which provides a brief overview of the project, including its objectives, scope, and key outcomes. This section allows the reader to quickly understand the purpose and significance of the system without going into technical details.

The **Introduction** section follows, presenting the background and context of the project. It explains the main problem addressed, defines the objectives, and identifies the target audience and end users. This section establishes the foundation for the rest of the report and highlights the importance of the system in an educational environment.

The **Methodology** section describes how the system was designed and developed. It explains the use of object-oriented programming principles, the design of the database, and the tools and technologies used in the implementation. This section also outlines the overall system architecture and workflow, showing how different components interact with each other.

The **System Implementation** section provides detailed technical information about how the system was built. It includes explanations of the Java classes, database structure, and rule-based recommendation logic. This section demonstrates how theoretical concepts are applied in practice to create a functional system.

The **Work in Progress** section describes the tasks that have been completed so far. It explains the development stages, the objectives of each task, and the results achieved. This section also discusses challenges encountered during development and the solutions applied to overcome them.

The **Lessons Learned** section presents the knowledge and skills gained throughout the project. It reflects on important insights related to programming, database management, and system design.

This section highlights how the project contributed to the development of practical and analytical skills.

The **Critical Self-Reflection** section focuses on personal evaluation. It analyzes individual contributions, identifies strengths and weaknesses, and discusses areas for improvement. This section emphasizes the learning experience and personal growth achieved during the project.

The **Future Work** section outlines possible improvements and extensions of the system. It suggests new features, technical enhancements, and ways to expand the system in future versions, demonstrating the scalability and potential of the project.

Finally, the report includes **References** and **Appendices**, where all external sources, technical documentation, diagrams, and additional materials are provided. These sections support the content of the report and ensure transparency and completeness.

Overall, this structured organization ensures that the report is clear, coherent, and easy to follow, allowing the reader to fully understand both the development process and the final outcome of the project.

Team Structure and Individual Contributions

Throughout the development of the project, teamwork was characterized by clear communication, shared responsibility, and continuous coordination between different parts of the system. Each member focused on specific tasks, but collaboration remained essential to ensure that all components—such as the Java object-oriented structure and the SQL database—were correctly integrated and functioned as a complete system.

One of the most important aspects of the collaboration was the ability to discuss ideas and make decisions together. We regularly reviewed system design choices, including class structure, database schema, and rule-based logic. This allowed us to identify potential issues early and improve the overall quality of the system. By sharing feedback and suggestions, we were able to refine both the technical implementation and the organization of the project.

Another key strength of the collaboration was flexibility. Although responsibilities were divided, we supported each other when challenges arose, especially during debugging and system integration. For example, aligning the database structure with the Java classes required careful coordination, and working together helped ensure consistency between different components.

The collaboration also played an important role in improving the clarity and quality of the final report. By reviewing each other's work and providing constructive feedback, we were able to present the project in a more structured and professional way. This process helped strengthen both our technical understanding and our ability to communicate complex ideas clearly.

Overall, the teamwork experience demonstrated the importance of communication, adaptability, and shared effort in software development. By working together effectively, we were able to develop a functional and well-organized system that integrates object-oriented programming, database management, and rule-based logic. This experience provided valuable skills that will be useful in future academic and professional projects.

Contributions of Margarita Sargsyan

Margarita focused mainly on the technical development of the system, particularly the implementation of object-oriented programming concepts in Java and the integration with the SQL database. She was responsible for designing the core system architecture, including creating classes such as Student, Course, Performance, and RecommendationEngine. These classes were used to structure the system in a modular and organized way.

In addition, Margarita worked on developing the rule-based logic used to analyze student performance. Instead of machine learning methods, she implemented predefined conditions to identify weak areas and generate recommendations. She also contributed to writing SQL queries for retrieving and managing student data, ensuring efficient interaction between the application and the database.

Furthermore, Margarita participated in testing and debugging the system to ensure correct functionality. She also contributed to writing the technical sections of the report, explaining the system design, implementation process, and key decisions.

Contributions of Mariam Harutyunyan

Mariam focused on the system design and overall structure of the application, ensuring that all components work together in a logical and efficient way. She contributed to planning the workflow of the system, from data input and storage to recommendation output.

Mariam also worked on organizing the database structure, including defining tables and relationships in SQL Server. She ensured that the data was stored in a clear and structured format, which supports efficient data retrieval and processing.

In addition, Mariam contributed to improving the usability of the system by organizing how results and recommendations are presented. She focused on making the system simple and understandable for users.

She also contributed to writing the descriptive and contextual parts of the report, including the introduction, target audience, and explanation of how the system can be used in educational environments.

Working Together

Throughout the development of the project, collaboration was based on regular communication, clear task distribution, and mutual support. Although each member had specific responsibilities, we continuously shared ideas, discussed design decisions, and reviewed each other's work to ensure consistency and quality across all components of the system.

The development process required close coordination between different parts of the system, including object-oriented programming in Java and database design using SQL Server. This made collaboration especially important, as decisions in one part of the system directly affected others. For example, the structure of the database needed to align with the design of the Java classes, and the rule-based logic had to correctly process the stored data. By working together, we ensured that all components were well integrated and functioned correctly.

We also collaborated during the testing and debugging phases of the project. By reviewing the system together and checking different scenarios, we were able to identify errors and improve the overall performance of the system. This process helped ensure that the recommendations generated by the system were accurate and consistent.

In addition, we worked together on the documentation and report writing. We reviewed each section, provided feedback, and made improvements to ensure that the report was clear, structured, and professionally written. This collaborative effort helped improve both the technical quality of the system and the clarity of its presentation.

Overall, teamwork played an important role in the success of the project. It allowed us to combine our knowledge, support each other during challenges, and produce a well-structured and functional

system. The experience highlighted the importance of communication, coordination, and shared responsibility in software development projects.

Contribution matrix detailing each participant's responsibilities and contributions

Task Number	Task Name	Objective	Description	Assigned Member	Status	Challenges Faced	Source
1	System Requirements Definition	Define how student data would be collected, stored, and processed.	Identifying main functionalities including key inputs like quiz scores and expected outputs like performance analysis.	Mariam Harutyunyan	Completed	Not in source	[1]
2	System Design Using Object-Oriented Programming	Create a modular and maintainable structure.	Designing architecture by creating Java classes such as Student, Course, Performance, and RecommendationEngine with specific attributes.	Margarita Sargsyan	Completed	Ensuring consistency between Java classes and database table columns.	[1]
3	Database Design and Implementation	Create a structured data storage system.	Designing relational schema in SQL Server, defining tables and relationships, and writing queries for data management.	Mariam Harutyunyan	Completed	Designing effective conditions for logic and ensuring normalization.	[1]
4	Integration of Java and Database	Enable communication between the application and the database.	Connecting the Java application with the SQL Server database using JDBC to allow dynamic data retrieval and processing.	Margarita Sargsyan	Completed	JDBC connectivity issues due to incorrect configuration and connection parameters.	[1]
5	Development of Rule-Based Recommendation Logic	Analyze student performance using predefined rules.	Implementing logic within the RecommendationEngine class to identify weak areas based on score thresholds and generate feedback.	Margarita Sargsyan	Completed	Defining conditions that are both simple and meaningful to produce accurate results.	[1]
6	Testing and Debugging	Ensure all components function correctly together.	Systematic verification of data processing, SQL queries, and recommendation outputs under different scenarios.	Margarita Sargsyan	Completed	Data consistency issues between object-oriented design and database structure.	[1]

The responsibilities were divided based on individual strengths. Margarita focused on the programming and logic implementation, while Mariam concentrated on database design and system structure. This distribution allowed efficient development and ensured that all components of the system were properly implemented and integrated.

Reflections on teamwork and collaboration dynamics

Throughout the development of the project, teamwork played a crucial role in ensuring the successful completion and integration of all system components. The collaboration was characterized by continuous communication, well-defined distribution of responsibilities, and a high level of mutual support. While each participant was assigned specific tasks based on individual strengths, ongoing coordination was essential to maintain coherence and consistency across the entire system.

A key aspect of the collaboration was the alignment of technical decisions. Since the project combined object-oriented programming in Java with relational database design in SQL Server, it was necessary to ensure that the system architecture remained consistent at both the application and database levels. For instance, the design of core classes such as *Student*, *Course*, and *Performance* needed to correspond directly to the database schema, including table structures and relationships. Regular discussions and joint evaluations of design choices helped prevent inconsistencies and improved the overall integrity of the system.

Another important element of the collaboration was the process of peer review and iterative improvement. By continuously sharing progress and providing constructive feedback, it was possible to identify potential issues at an early stage and refine both the technical implementation and system design. This approach proved particularly valuable during the integration phase, where different modules had to interact seamlessly. Collaborative testing and debugging ensured that the system produced reliable and consistent results under various conditions.

Flexibility and adaptability also played a significant role in the effectiveness of the teamwork. Although responsibilities were initially divided, both participants remained open to supporting each other when challenges emerged. For example, difficulties related to database queries or system logic were addressed collectively, leading to more efficient problem-solving and a deeper shared understanding of the system. This cooperative approach contributed to maintaining steady progress and minimizing potential delays.

In addition to the technical dimension, collaboration significantly enhanced the quality of the project documentation. Through joint review and revision of report sections, the team ensured that the content was logically structured, clearly articulated, and professionally presented. This not only improved readability but also strengthened the overall academic quality of the work.

In conclusion, the teamwork experience highlighted the importance of effective communication, coordination, and shared accountability in software development projects. The ability to collaborate efficiently allowed for the successful integration of object-oriented programming, database management, and rule-based system design. Moreover, this experience contributed to the development of essential professional skills, including critical thinking, adaptability, and collaborative problem-solving, which are highly valuable in both academic and professional contexts.

Methodology

Project Planning and Development Process

The development of the Student Performance Tracking and Recommendation System followed a structured and iterative approach inspired by Agile methodology. Instead of implementing the

entire system at once, the project was divided into smaller phases, allowing continuous improvement, testing, and refinement at each stage. This approach ensured flexibility and made it possible to identify and resolve issues early in the development process.

The first phase involved defining the system requirements and identifying the core functionalities. This included determining how student data would be collected, stored, and processed, as well as how recommendations would be generated based on predefined rules. During this stage, special attention was given to the overall system architecture, ensuring that the design would be modular and scalable.

The second phase focused on system design. Object-oriented programming principles were applied to structure the system into logical components. Classes such as *Student*, *Course*, *Performance*, and *RecommendationEngine* were designed to represent different entities within the system. At the same time, the relational database schema was defined, including tables, attributes, and relationships. This ensured consistency between the application layer and the database layer.

The third phase consisted of implementation. The system was developed using Java for the application logic and SQL Server for database management. During this phase, the database was created, data was inserted and retrieved using SQL queries, and rule-based logic was implemented to analyze student performance. Each component was developed and tested individually before being integrated into the full system.

The final phase involved testing and validation. Different scenarios were used to verify the correctness and reliability of the system. The outputs were evaluated to ensure that the recommendations were accurate and consistent with the input data. This iterative process of testing and refinement contributed to improving the overall quality and stability of the system.

Tools and technologies used

The development of the system required the use of multiple tools and technologies, each playing a specific role in the implementation process.

Java was used as the primary programming language for implementing the application logic. Its support for object-oriented programming allowed the system to be structured into modular and reusable components. This improved code organization, readability, and maintainability.

SQL Server was used as the database management system for storing and managing student data. It provided a structured environment for organizing information into tables and allowed efficient data retrieval through SQL queries. The use of relational databases ensured data consistency and integrity.

JDBC (Java Database Connectivity) was used to connect the Java application with the SQL database. This enabled the system to perform operations such as inserting, updating, and retrieving data directly from the database.

Additional tools included development environments such as **IntelliJ IDEA** or **Eclipse**, which were used for writing, testing, and debugging the Java code. Version control tools such as **GitHub** were used to manage project files and track changes throughout development.

Overall, the selected technologies ensured that the system was robust, efficient, and aligned with modern software development practices.

ADA Compliance Considerations and Implementation

Accessibility was an important consideration in the design of the system. The project aimed to follow key principles of the Americans with Disabilities Act (ADA) to ensure that the system can be used by individuals with different abilities. Although the system is a prototype, several accessibility features were taken into account during development.

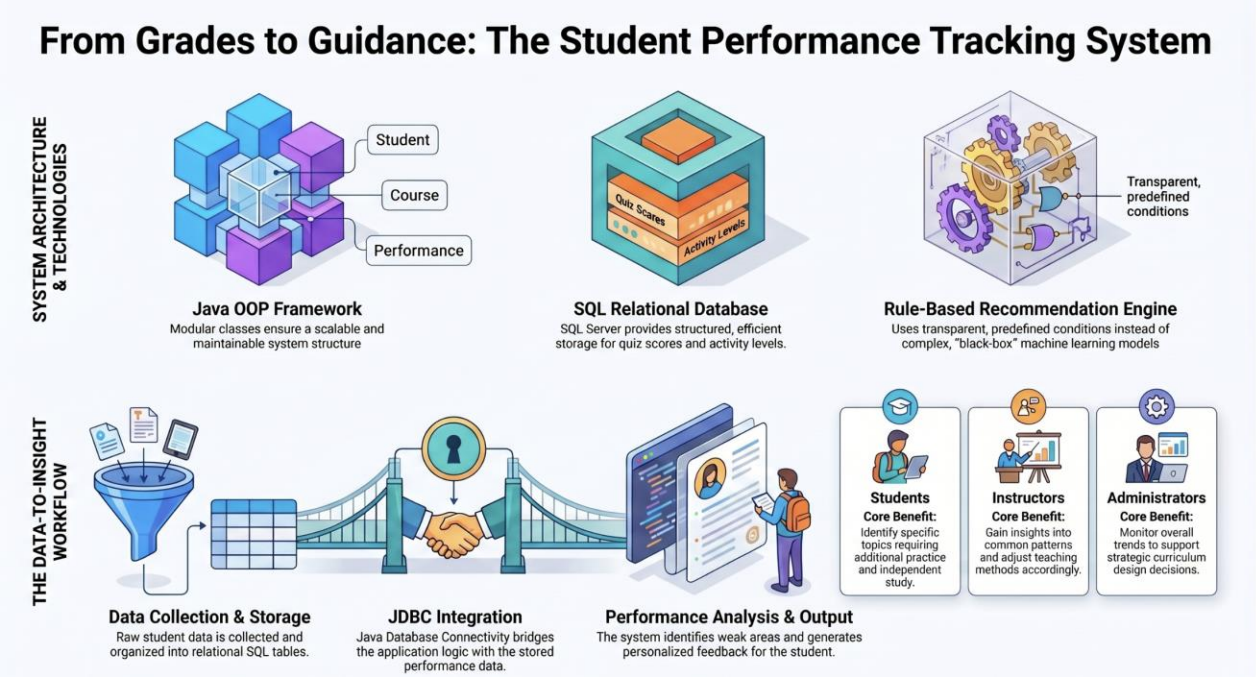
One important aspect was **clear and readable interface design**. Text elements were structured in a way that ensures readability, using simple language and logical organization. This helps users with cognitive difficulties better understand the information presented by the system.

Another consideration was **color contrast and visual clarity**. The system design avoids relying only on color to convey important information, ensuring that users with visual impairments or color blindness can still interpret the results correctly.

The system also supports **logical navigation and structured layout**, making it easier for users to interact with the system without confusion. This is especially important for users who rely on assistive technologies such as screen readers or keyboard navigation.

Additionally, input fields and outputs are clearly labeled, ensuring that users understand how to interact with the system and interpret the results. This improves usability and reduces the risk of errors.

Although the current implementation is a prototype, these considerations demonstrate an effort to create an inclusive and accessible system. Future improvements may include more advanced accessibility features, such as full screen reader compatibility and adaptive interface design.



Research and Analysis

Literature Review or Research Conducted to Support the Project

In order to support the development of the Student Performance Tracking and Recommendation System, a review of existing research and practices in educational technology and software engineering was conducted. The analysis focused on how student performance data can be effectively managed, structured, and utilized to improve learning outcomes without relying on complex computational models.

Academic and technical resources emphasize the importance of structured data management in educational systems. Studies highlight that organizing student data using relational databases significantly improves data consistency, accessibility, and scalability. This directly influenced the decision to use SQL Server as the core data storage solution, ensuring that student performance information is stored in a structured and efficient manner.

Additionally, research in software engineering demonstrates that object-oriented programming is one of the most effective approaches for designing modular and maintainable systems. Concepts such as encapsulation, abstraction, and modular design allow developers to separate system

components into clearly defined classes. This approach improves system organization and simplifies future modifications. As a result, the project adopted a Java-based object-oriented architecture to represent system entities such as students, courses, and performance records.

Furthermore, literature on rule-based systems indicates that predefined logical conditions can be effectively used for decision-making processes in cases where transparency and simplicity are required. Unlike more complex systems, rule-based approaches allow users to clearly understand how outputs are generated. This was particularly important for this project, as the goal was to create a system that provides understandable and explainable recommendations.

Overall, the research phase provided a strong theoretical foundation for selecting appropriate technologies and design approaches, ensuring that the system is both practical and aligned with established principles in computer science.

Comparative Analysis of Similar Systems or Solutions

As part of the research process, several existing systems and approaches were analyzed to understand their strengths and limitations. Many modern educational platforms, such as learning management systems (LMS), provide basic features for tracking student performance, including grades, attendance, and activity records. However, these systems often focus primarily on data storage and presentation, rather than meaningful analysis or recommendation generation.

More advanced systems attempt to provide personalized feedback and adaptive learning pathways. While these systems can be effective, they often rely on complex algorithms or require large datasets, making them difficult to implement in smaller-scale or academic projects. Additionally, such systems may lack transparency, as users do not always understand how recommendations are generated.

In contrast, the system developed in this project adopts a simpler and more transparent approach. By using rule-based logic, the system generates recommendations based on clearly defined

conditions, such as score thresholds. This makes the system easier to understand, implement, and maintain.

Another point of comparison involves system architecture. Many existing solutions integrate multiple technologies, but may lack modular design. In this project, the use of object-oriented programming ensures that the system is well-structured, with clear separation between components such as data management, logic processing, and output generation. This improves maintainability and scalability.

Overall, the comparative analysis highlights that while complex systems offer advanced features, a well-designed rule-based system combined with strong software engineering principles can provide an effective and practical solution.

Challenges and Insights Gained from the Research Phase

The research phase revealed several challenges that influenced the design and implementation of the system. One of the main challenges was determining how to design a system that is both effective and simple. While many advanced solutions exist, they often require complex algorithms and large datasets, which are not always suitable for academic projects. This led to the decision to adopt a rule-based approach, which balances functionality with simplicity and transparency.

Another challenge involved structuring the data efficiently. Designing a relational database required careful consideration of how different entities—such as students, courses, and performance records—should be connected. This process emphasized the importance of normalization and proper relationship design in database systems.

From a system design perspective, integrating object-oriented programming with database management presented additional challenges. Ensuring that Java classes accurately represent

database tables required careful planning and consistency. This highlighted the importance of aligning application-level design with database structure.

The research phase also provided important insights into the role of clarity and usability in system design. It became clear that a system does not need to be overly complex to be effective. Instead, simplicity, transparency, and logical structure are key factors in creating a useful application.

Overall, the challenges encountered during the research phase contributed to a deeper understanding of software design principles and helped guide the development of a system that is both practical and well-structured.

Work in Progress

Detailed description of tasks completed so far

The development of the Student Performance Tracking and Recommendation System has progressed through several clearly defined stages. Each task was completed with specific objectives, ensuring a structured and systematic development process.

Task 1: System Requirements Definition

The first step involved identifying the main functionalities of the system. The objective was to define how student data would be collected, stored, and processed. This included determining the key inputs (such as quiz scores and assignments) and expected outputs (performance analysis and recommendations). This stage provided a clear roadmap for the rest of the project.

Task 2: System Design Using Object-Oriented Programming

The second task focused on designing the system architecture using Java object-oriented programming principles. The objective was to create a modular and maintainable structure. Classes such as *Student*, *Course*, *Performance*, and *RecommendationEngine* were defined. Each class was designed with specific attributes and methods to represent real-world entities and their interactions within the system.

Task 3: Database Design and Implementation

The third task involved designing and implementing a relational database using SQL Server. The objective was to create a structured data storage system. Tables were created for students, courses, and performance records, with clearly defined relationships. SQL queries were written to insert, retrieve, and manage data efficiently.

Task 4: Integration of Java and Database

The fourth stage focused on connecting the Java application with the SQL database using JDBC. The objective was to enable communication between the application and the database. This allowed the system to retrieve student data, process it, and generate outputs dynamically.

Task 5: Development of Rule-Based Recommendation Logic

The next task was to implement the core functionality of the system: the recommendation engine. The objective was to analyze student performance using predefined rules. For example, if a student's quiz score falls below a certain threshold, the system generates a recommendation to review the corresponding topic. This logic was implemented within the *RecommendationEngine* class.

Task 6: Testing and Debugging

The final completed task involved testing the system under different scenarios. The objective was to ensure that all components function correctly together. Various test cases were used to validate data processing, database queries, and recommendation outputs.

Challenges Faced and Solutions Applied

During the development process, several challenges were encountered. One of the main challenges was ensuring consistency between the Java classes and the database structure. Since both components represent the same data, any mismatch could cause errors. This issue was resolved by carefully aligning class attributes with database table columns.

Another challenge involved establishing a stable connection between Java and SQL Server using JDBC. Initial errors occurred due to incorrect configuration and connection parameters. This was resolved by adjusting the connection settings and testing the communication step by step.

Designing effective rule-based logic was also challenging. The difficulty was in defining conditions that are both simple and meaningful. This was addressed by testing different scenarios and refining the rules to produce more accurate and useful recommendations.

Additionally, debugging the system required careful analysis of both code and database queries. By systematically testing each component, it was possible to identify and fix errors efficiently.

Code Highlights Linked to Git Repository with Explanations for Critical Parts

Several important parts of the system code play a key role in its functionality. One of the most critical components is the database connection module, which establishes communication between the Java application and the SQL Server database. This ensures that data can be retrieved and updated in real time.

Another important component is the *RecommendationEngine* class. This class contains the rule-based logic used to analyze student performance. It processes input data and generates recommendations based on predefined conditions. This is the core functionality of the system.

Additionally, SQL queries used for data retrieval are essential for system performance. Efficient queries ensure that data is processed quickly and accurately.

The project repository includes all source code, allowing for further review and extension of the system. Each critical part of the code has been structured and documented to ensure clarity and maintainability.

Lessons Learned

Insights Gained from Each Completed Task

The development of the Student Performance Tracking and Recommendation System provided valuable insights at each stage of the project. During the system planning phase, it became clear that defining clear requirements is essential for guiding the entire development process. A well-structured plan reduces confusion and helps maintain consistency across different components of the system.

During the design phase, the use of object-oriented programming highlighted the importance of modularity and abstraction. Creating separate classes for different system entities improved code organization and made the system easier to understand and maintain. This demonstrated how OOP principles contribute to building scalable and flexible applications.

The database design phase emphasized the importance of structuring data correctly. Proper normalization and clear relationships between tables ensured data consistency and reduced redundancy. This experience reinforced the role of relational databases in managing structured information efficiently.

During implementation, integrating Java with the SQL database provided insight into how different technologies interact within a system. It showed the importance of maintaining consistency between application logic and data storage.

Finally, the testing phase demonstrated the importance of verifying system functionality under different conditions. Even small errors in logic or data handling can affect the overall system output, making thorough testing a critical part of development.

Challenges Addressed and Lessons for Future Tasks

Several challenges encountered during the project provided important learning opportunities. One of the main challenges was ensuring consistency between the object-oriented design and the database structure. Differences between class attributes and database fields initially caused errors, highlighting the importance of careful planning and alignment between system components.

Another challenge was establishing a stable connection between Java and the SQL database. Configuration issues and connection errors required detailed debugging. This experience emphasized the importance of understanding system integration and testing connections step by step.

Defining effective rule-based logic also presented difficulties. It required careful consideration to ensure that the conditions were both meaningful and practical. Through testing and refinement, the logic was improved to produce more accurate recommendations. This demonstrated that even simple systems require thoughtful design to be effective.

From these challenges, an important lesson for future work is the need for incremental development and continuous testing. Breaking the system into smaller components and verifying each part individually helps reduce errors and simplifies debugging. Additionally, planning extra time for testing and refinement is essential for achieving reliable results.

Reflections on the Effectiveness of Chosen Tools and Methods

The tools and methods selected for this project proved to be effective in achieving the desired outcomes. Java, as a programming language, provided strong support for object-oriented design, enabling the creation of a well-structured and maintainable system. Its flexibility and wide range of libraries made it suitable for implementing application logic.

SQL Server was an appropriate choice for database management, as it allowed efficient storage and retrieval of structured data. The use of SQL queries ensured fast data processing and reliable performance. The integration of Java and SQL through JDBC demonstrated the effectiveness of combining programming and database technologies in a single system.

The decision to use a rule-based approach instead of more complex methods proved to be practical and efficient. This approach ensured transparency, as the logic behind recommendations is clear and easy to understand. It also simplified implementation and reduced computational complexity.

Overall, the chosen tools and methods provided a solid foundation for the system. They allowed the project to achieve its objectives while maintaining simplicity, clarity, and efficiency. This experience demonstrated that well-selected technologies and properly applied design principles are key to building effective software systems.

Final Deliverables and Achievements

Overview of the Developed Software

The final outcome of this project is a functional prototype of a Student Performance Tracking and Recommendation System developed using Java object-oriented programming and SQL database technologies. The system is designed to manage and analyze student academic data, including quiz scores, assignment results, and activity levels, and to generate recommendations based on predefined rules.

The software consists of several interconnected components. The application layer is implemented in Java, where object-oriented principles are used to structure the system into modular classes such as *Student*, *Course*, *Performance*, and *RecommendationEngine*. These classes handle data processing, system logic, and interaction between components.

The data storage layer is implemented using SQL Server, where all student-related information is stored in structured tables. Relationships between tables ensure data consistency and allow efficient retrieval of information. The integration between Java and SQL is achieved through JDBC, enabling real-time communication between the application and the database.

The system successfully demonstrates how student performance data can be processed and transformed into meaningful recommendations. It provides a clear example of how fundamental computer science concepts—such as object-oriented programming, database management, and rule-based logic—can be applied to develop a practical software solution.

Instructions for Running the Code and Accessing the Repository

The complete source code of the project is stored in a version-controlled repository, allowing easy access and management of all project files. The repository includes Java source files, database scripts, and documentation necessary to run the system.

To run the system, the following steps should be followed:

1. Install a Java development environment (such as IntelliJ IDEA or Eclipse) and ensure that the Java Development Kit (JDK) is properly configured.
2. Install and configure SQL Server, and execute the provided SQL scripts to create the database and required tables.
3. Import the project into the development environment and configure the JDBC connection settings, including database URL, username, and password.
4. Compile and run the main Java application file.
5. Input or retrieve student data from the database and observe the generated recommendations.

The repository also contains configuration files and comments within the code to guide users through the setup process. This ensures that the system can be easily replicated and tested in different environments.

User Guide or Demonstration of Software Functionality

The system is designed to be simple and intuitive, allowing users to interact with it without requiring advanced technical knowledge. The workflow of the system follows a clear sequence of steps.

First, student performance data is entered into the system, either manually or through database insertion. This data includes key indicators such as quiz scores, assignment results, and activity levels.

Next, the system processes the data using the Java application. The data is retrieved from the SQL database and passed through the rule-based logic implemented in the *RecommendationEngine*. Based on predefined conditions, the system identifies weak areas in student performance.

After processing, the system generates recommendations. These recommendations provide clear guidance on which topics the student should review or improve. The output is presented in a structured and understandable format, ensuring that users can easily interpret the results.

The overall workflow can be summarized as follows:

- Input student data
- Store and retrieve data from the database
- Analyze performance using rule-based logic
- Generate and display recommendations

This demonstration confirms that the system successfully achieves its objective of transforming raw academic data into actionable insights. The simplicity and clarity of the system make it suitable for educational environments and future extensions.

Critical Self-Reflection

Individual Reflections on Task Contributions

What went well?

One of the most successful aspects of this project was the ability to design and implement a complete system independently. The use of object-oriented programming in Java allowed the system to be structured in a clear and modular way, making it easier to manage and extend. Each component, including data processing and recommendation logic, was well-organized and functioned as expected.

Another important achievement was the successful integration of the Java application with the SQL database. The system was able to store, retrieve, and process student data efficiently, demonstrating a strong understanding of how different technologies interact within a software system.

Additionally, the development process was carried out in a structured manner. Tasks were completed step by step, which helped maintain consistency and reduced errors during implementation. Overall, the system achieved its main objectives and provided a functional and reliable solution.

What could have been improved?

Despite the successful completion of the project, there are several areas that could have been improved. One of the main challenges was time management, particularly during the implementation and debugging phases. Some tasks required more time than initially expected, which created pressure toward the final stages of development. In future projects, better planning and time allocation would improve efficiency.

Another area for improvement is system optimization. While the system functions correctly, further enhancements could be made to improve performance, especially when handling larger

datasets. This includes optimizing SQL queries and refining the logic to handle more complex scenarios.

Additionally, the user interaction aspect of the system could be further developed. Improving the interface and making it more interactive would enhance usability and provide a better user experience.

Skills gained or developed during the project?

This project contributed significantly to the development of both technical and analytical skills. One of the most important skills gained was a deeper understanding of object-oriented programming in Java, including how to design classes, manage data, and structure a complete system.

Another key skill developed was database management using SQL. The project provided practical experience in designing relational databases, writing queries, and managing structured data efficiently.

In addition, the project strengthened problem-solving and debugging skills. Identifying errors, analyzing their causes, and finding effective solutions required logical thinking and persistence.

Finally, the project improved the ability to work independently, manage complex tasks, and organize technical information in a clear and professional way. These skills are essential for future academic and professional work in the field of computer science.

Reflection on the Group's Collaborative Efforts and Outcomes

Although this project was primarily completed as an individual assignment, reflecting on collaboration remains important in the context of software development practices. Even in individual work, different components of the system must function together in a coordinated way, similar to how team members collaborate in a group project.

The development process required the integration of multiple elements, including Java object-oriented programming, SQL database management, and rule-based logic. Each of these components had to align correctly to ensure the overall functionality of the system. This experience demonstrated that effective “internal collaboration” between system components is just as important as teamwork between individuals.

From a broader perspective, previous teamwork experiences influenced the way this project was approached. Skills such as organizing tasks, maintaining consistency, and thinking about system structure from multiple perspectives contributed to a more efficient and structured workflow.

The final outcome of the project reflects successful coordination between all system components. The application is functional, well-structured, and capable of performing its intended tasks effectively. This demonstrates the ability to manage complex processes independently while applying principles commonly used in collaborative environments.

Overall, this experience highlights the importance of coordination, structure, and consistency in achieving successful outcomes. Whether working individually or in a group, these factors play a critical role in the development of reliable and well-designed software systems.

Future Work

Although the Student Performance Tracking and Recommendation System has been successfully developed and demonstrates its core functionality, there are still several areas that can be further improved and expanded. As with most software projects, the current version represents an initial prototype rather than a fully optimized or scalable solution.

This section outlines the key limitations of the system, identifies unresolved challenges, and presents potential directions for future development. It also includes suggestions for improving the overall development process, based on the experience gained during the project.

Unresolved Issues or Challenges

Despite the successful development of the Student Performance Tracking and Recommendation System, several limitations and unresolved challenges remain. One of the main issues is the scalability of the current implementation. The system is designed as a prototype and has been tested on relatively small datasets. Handling larger volumes of data may require optimization of both the application logic and database queries to ensure consistent performance.

Another challenge involves system flexibility. The current rule-based logic is effective for simple scenarios but may not fully capture more complex patterns in student performance. Expanding the logic to handle a wider range of academic situations would improve the system's overall usefulness.

Additionally, the user interface remains relatively basic. While the system is functional, improving the presentation and interaction design would enhance usability and make the system more accessible to a broader range of users.

Finally, integration with external systems, such as learning management platforms, has not yet been implemented. This limits the system's ability to operate in real-world educational environments.

Features or Enhancements Planned for the Future

Several enhancements can be introduced in future versions of the system to improve its functionality and performance. One important improvement is the development of a more advanced user interface, possibly in the form of a web-based application. This would allow users to interact with the system more easily and access it from different devices.

Another planned enhancement is the expansion of the rule-based recommendation system. Additional conditions and more detailed analysis could be introduced to provide more personalized and precise recommendations.

Future versions may also include user account management, allowing students to track their performance over time. This would transform the system from a simple prototype into a continuous monitoring tool.

Furthermore, integration with external educational platforms or databases could be implemented. This would enable real-time data collection and make the system more practical for real-world use.

Additional technical improvements may include optimizing database queries, improving system performance, and enhancing data security measures.

Suggestions for Improving the Development Process

Reflecting on the development process, several improvements can be suggested for future projects. One key recommendation is the adoption of a more formal project management approach, such as Agile methodology with clearly defined iterations and milestones. This would help ensure better organization and progress tracking.

Another important improvement is the inclusion of earlier and more frequent testing phases. Instead of testing mainly at the end of development, incorporating testing throughout the process would help identify issues sooner and reduce the time required for debugging.

Improved documentation practices are also essential. Writing documentation alongside development, rather than at the end, would ensure greater accuracy and reduce workload during the final stages of the project.

Finally, allocating more time for system optimization and refinement would enhance the overall quality of the final product. Planning buffer time for unexpected challenges is an important strategy for managing complex software projects effectively.

Overall, these improvements would lead to a more efficient, structured, and high-quality development process in future projects.

References

The following sources were used to support the theoretical background and technical implementation of the project. All references are presented in a consistent academic format.

- Oracle Java
- Microsoft SQL Server Documentation
- JDBC (Java Database Connectivity) Guide
- Database Design Basics (Microsoft)
- Software Engineering Concepts (IEEE Overview)
- Object-Oriented Programming Concepts (GeeksforGeeks)

Appendices

GitHub link: